EngSci 721

Inverse Problems and Learning From Data

Oliver Maclaren (oliver.maclaren@auckland.ac.nz)

Module overview

Inverse Problems and Learning From Data (*Oliver Maclaren*)
[~14 lectures/3 tutorials]

1. *Basic concepts* [5 lectures + 1 Tutorial]

   Forward vs inverse problems. Well-posed vs ill-posed problems. Algebra and calculus of inverse problems (left and right inverses, generalised and pseudo inverses, resolution operators, matrix calculus). Representing higher dimensional problems (image data etc).

2. *Instability and regularisation in linear and nonlinear problems* [6 lectures + 1 Tutorial]

   Instability and related issues for generalised inverses. Introduction to regularisation and trade-offs. Tikhonov regularisation. Higher-order Tikhonov regularisation. Sparsity and regularisation using different norms. Truncated singular value decomposition. Iterative regularisation, including stochastic/mini-batch gradient descent.

3. *Further topics* [3 lectures + 1 Tutorial]

   Regularisation parameter choice, including statistical and machine learning views of regularisation. Confidence sets for linear and nonlinear models. Physics-informed machine learning and neural networks.

Lecture 13: Regularisation parameter choice

Topics:

- Regularisation paths and parameter-selection questions
- Discrepancy principles
  - Noise models, confidence regions, and compatible solutions
- Cross-validation
  - Prediction of held-out data

EngSci 721: Lecture 13

Regularisation parameter choice methods

From L-curve to

- discrepancy principle
- cross-validation / out of sample prediction

→ both are 'statistical' cf 'pragmatic' L-curve
↳ pros & cons!

Further ideas: double descent??

Warning

- Regularisation parameter selection methods are 'easy' to implement but subtle to interpret

- Only expect you to describe basic ideas & how would implement; not details

- Will use some random variable stuff but RV will cover 'properly' later

---

we are going to consider two more regularisation parameter selection methods

- discrepancy principle
- cross-validation

→ here in context of Tikhonov regularisation, but (like L-curve) they generalise to other regularisation methods (eg when to stop, how many singular values etc)

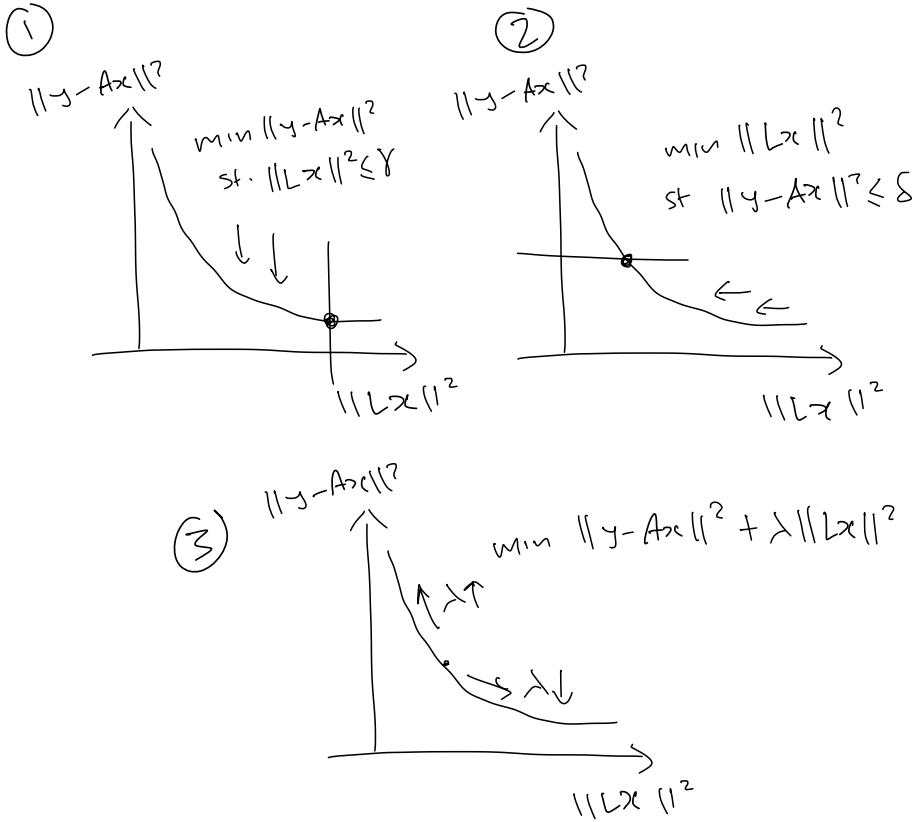
→ in machine learning often call λ etc a 'hyperparameter'

↳ a parameter of the method of estimation

↳ can also link to a property of the model itself, sometimes

→ 'statistical': based on 'data as random' ideas

Recall three ways to think about trade-off curves:



These are 'equivalent' in the sense that they give the same soln for appropriate choice of λ, δ, γ

(under appropriate regularity, convexity etc assumptions)

Discrepancy principles

The general idea of a discrepancy principle is based on (2):

$$\begin{cases} \min \|Lx\|^2 & \} \text{ model norm term} \\ \text{s.t. } \|y - Ax\|^2 \le \delta & \} \text{ data norm term} \end{cases}$$

for some choice of δ

Assuming the constraint is binding this means we enforce $\|y - Ax\|^2 = \delta$ for some δ.

Different methods of choosing δ exist. Here is one that has a nice interpretation (imo!)

↳ 'simplest adequate/acceptable fitting model'

Discrepancy: first consider data fit

Goal is to 'solve' '$y \approx Ax$'

ie $\begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_m \end{bmatrix} \approx \begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ & & \vdots & \\ a_{m1} & & \cdots & a_{mn} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}$

Now:

- Suppose 'true' value is x^*
- Fix locations we measure at $(1, \dots m)$ but imagine repeated measurements & that measured data has IID normal 'random noise'. Statistical model:

$$\boxed{y = Ax^* + \varepsilon, \quad \varepsilon \sim N(0, \sigma^2 I_m)}$$

ie $\begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_m \end{bmatrix} = \begin{bmatrix} a_{11} & \cdots & a_{1n} \\ & \vdots & \\ a_{m1} & \cdots & a_{mn} \end{bmatrix} \begin{bmatrix} x_1^* \\ x_2^* \\ \vdots \\ x_n^* \end{bmatrix} + \begin{bmatrix} \varepsilon_1 \\ \varepsilon_2 \\ \vdots \\ \varepsilon_m \end{bmatrix}$

Note: assume actual data generated from true but unknown x^*

Using error model assumption: assume σ known

→ we don't know x^* or realised value of error

→ we do know (by assumption) the distribution of error.

↳ what does data misfit look like?

First, need:

Chi Square:

If $z_i \overset{iid}{\sim} N(0,1)$ (standard normal)

Then $\boxed{Q = \sum_{i=1}^{k} z_i^2}$ } sum of squared errors

has chi-square / chi squared distribution with k degrees of freedom' & write $\boxed{Q \sim \chi^2_k}$

Then, (row i)

in model, $z_i = \dfrac{y_i - a_i^T x^*}{\sigma} \sim N(0,1)$

& $\dfrac{1}{\sigma^2}\|y - Ax^*\|^2 = \sum_{i=1}^{m} \left(\dfrac{y_i - a_i^T x^*}{\sigma}\right)^2$

$\sim \chi^2_m$

ie we know distribution of $\|y - Ax^*\|^2$ (given σ noise level)

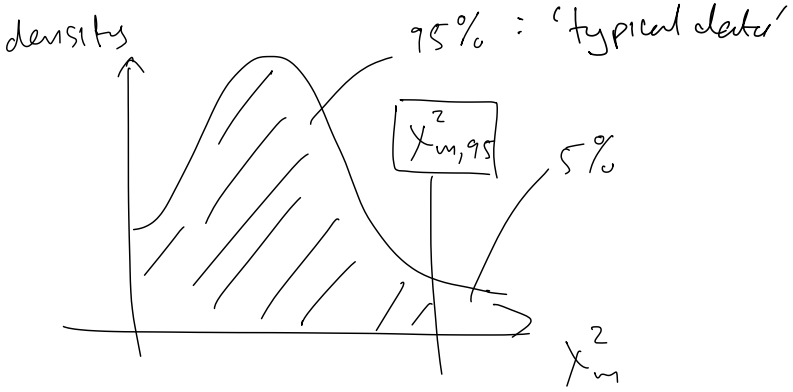
How do we use?

↳ match to mean or median, or

↳ use for confidence interval

Confidence interval via hypothesis test inversion

Chi square:



- 95% of time $\chi^2_m \leq \chi^2_{m,95}$ under x^*.
  - ↳ call 'typical data'
- Suppose y_{obs} is 'typical' under x^* } assump
- Consider y_{obs} under some candidate x
  - if not typical under x, & typical under x^*, then $x^* \neq x$. Rule out
  - collect all x such that data is typical
  - 95% of time this set of x will contain x^*, as 95% of time data will be typical under x^*.

Confidence set procedure

define procedure:

$$C_{95}(Y) = \left\{ x \;\middle|\; \frac{1}{\sigma^2}\|Y - Ax\|^2 \leq \chi^2_{m,95} \right\}$$

$$\Rightarrow \mathbb{P}\left(x^* \in C_{95}(Y)\right) = 0.95$$

↑ random due to new realisations

realised: $C_{95}(y_{obs})$.

procedure works 95% of time.

Back to:

$$\min \|Lx\|^2$$

$$\text{s.t. } \|y - Ax\|^2 \leq \delta.$$

Our confidence set determines

$$\|y - Ax\|^2 \leq \sigma^2 \chi^2_{m,95}$$

ie use $\boxed{\delta = \sigma^2 \chi^2_{m,95}}$

→ we could stop here and get a full set estimate

→ to get point estimate we now solve the $\min \|Lx\|^2$ over the confidence set.

→ if $x^* \in C$ then $\min \|Lx\|^2 \leq \|Lx^*\|^2$, ie lower confidence bound on true model complexity

↳ model can't be simpler (95% conf.)

↳ in general no upper bound

↳ model could be more complex.

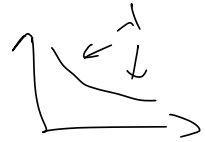
Solving?

- Can eg solve in form

$$\|y - Ax\|^2 + \lambda \|Lx\|^2$$

for grid of λ.

→ Traces curve.



Then find point for which $\|y - Ax\|^2 = \sigma^2 \chi^2_{m,95} = \delta$



- Or eg iterative min until $\|y - Ax\|^2 = \delta$.

→ gives lower bound

→ a more 'central' estimate uses eg $\chi^2_{m,50}$, but still tends to 'oversmooth' as a point estimate

---

Convenient: for large m,

$\chi_{m,50} \approx m$ (also mean)

so $\delta \sim \sigma^2 m$ reasonable (Morozov)

Example: polynomial ($x^{[q]}$) regression, known noise level ($\sigma = 1$)

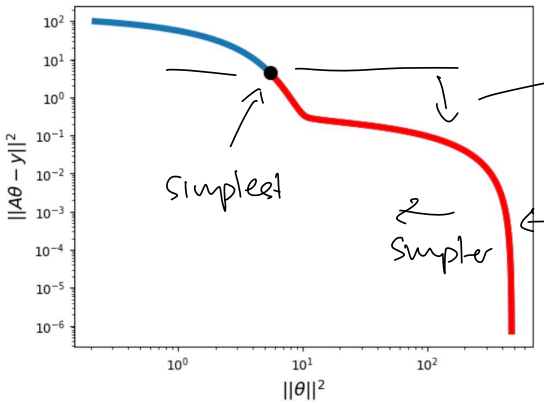
- Tikhonov $\lambda\|\theta\|^2$ ← note: use θ instead of x here
- Loop over each λ, calc norms
- Then:

median

```
from scipy.stats import chi2
delta = sigma**2*chi2.ppf(q=0.5,df=len(yp_obs))
indices = data_norms**2 <= delta
data_norms_delta = data_norms[indices]
theta_norms_delta = theta_norms[indices]
nreg = len(data_norms_delta)

plt.loglog(theta_norms**2,data_norms**2,linewidth=5)
plt.loglog(theta_norms_delta**2,data_norms_delta**2,'r',linewidth=5)
plt.loglog(theta_norms[nreg]**2,data_norms[nreg]**2,marker='o',markersize=10,linewidth=5,color='k')

plt.ylabel(r'$||A\theta-y||^2$',fontsize=14)
plt.xlabel(r'$||\theta||^2$',fontsize=14)
plt.show()
```

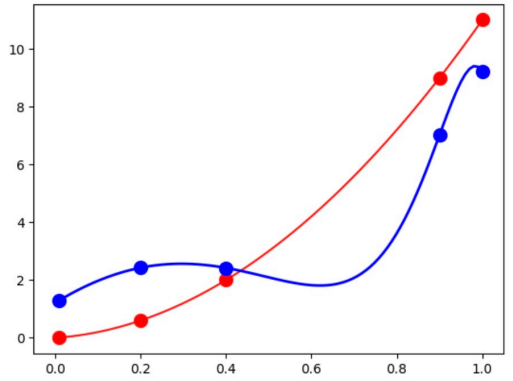


acceptable fit

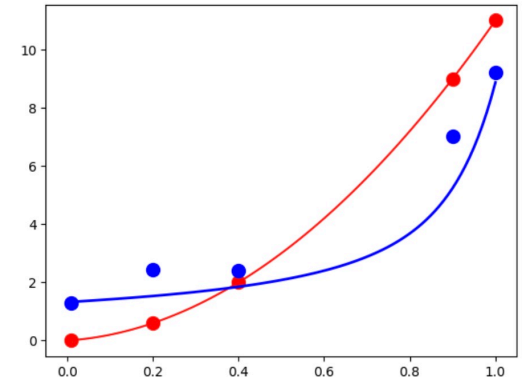
red: pareto path in confidence set for $\|\theta\|^2$!

Note: even median quite oversmoothed as point-estimate

Example cont'd



pseudo inverse sol (blue)



discrepancy median (over smoothed but treat as lower bound)

Cross-validation

- Discrepancy principles ask about which x are compatible with the data
  - ↳ eg simplest compatible
  - ↳ based on noise model with explicit σ (or equiv).
  - ↳ naturally related to set estimation (confidence intervals etc)

- Cross validation asks about predicting new data realisation
  - ↳ relates to estimating Ax^* rather than x^* as such
  - ↳ held out data plays role of noise model
  - ↳ naturally a point estimation problem
  - ↳ can still help with x^*, indirectly
    - ↳ better estimation of Ax^* helps (tho' not same as) est. of x^*.

Cross validation target (subtle!)

consider simple problem of predicting data realisation in scalar:

$$Y = u^* + \varepsilon, \quad \varepsilon \sim N(0, \sigma^2)$$

Y: random data; u^*: true mean; ε: random error

→ predict number a & evaluate expected error

Can show (dep on a.)

$$\mathbb{E}\left[(Y-a)^2\right] = \sigma^2 + (a-u^*)^2$$

($\mathbb{E}$ expectation operator; $\mathbb{E}[(Y-a)^2]$: expected error; σ^2: constant)

minimum for $a = u^*$

→ ie the mean is best (constant) prediction of new data. Don't know the

→ to predict we can try to estimate the mean. (& vice-versa)

Inverse problem (vector data)

In $y \approx Ax$

ie $Y = Ax + \mathcal{E}$, $\mathcal{E}$ mean zero error.

mean is $u = \mathbb{E}[Y] = \mathbb{E}[Ax + \mathcal{E}]$

$= Ax$

least squares estimate is

$\hat{u} = A\hat{x} = AA^{+}y = R_d y \quad (= H y$ ↑ hat matrix in stats)

$= y$ if underdet.

Can we do 'better'?

→ can enlarge class of estimators of u^*

→ evaluate 'expected loss'

↳ see my old 760 notes...

Estimator (estimator)

- in frequentist stat. theory define estimation rule for quantity & evaluate rule under repeated use (new Y)

Here consider Tikhonov as function of random data Y:

$$\hat{x}_\lambda(Y) = \operatorname{argmin} \|Y - Ax\|^2 + \lambda \|Lx\|^2$$

→ each Y realisation gives new x_λ estimate.

Use to predict mean:

$$\hat{u}_\lambda(Y) = A\hat{x}_\lambda(Y) \quad \text{ie } R_d^{*,\lambda} Y$$

Want to assess

$$M(\lambda) = \mathbb{E}\left[\|\hat{u}_\lambda(Y) - u^*\|^2\right]$$

ie expected squared error in est. u^*.

Problem: don't know u^*! 'validation'

- However, can show, for random $y^{val.}, y^{train}$

$$\underbrace{\mathbb{E}_{y^{train},\, y^{val}}\left[\| y^{val} - \hat{u}_\lambda(y^{train})\|^2\right]}_{②} = m\sigma^2 + \underbrace{\mathbb{E}_{y^{train}}\left[\|\hat{u}_\lambda(y^{train}) - u^*\|^2\right]}_{①}$$

$m\sigma^2$ ~ constant

want to min ① ⇒ train best model

can estimate via ② if have additional validation data.

→ $y^{val.}$ known

→ $\hat{u}_\lambda(y^{train})$ known

$\arg\min$ ② = $\arg\min$ ①

Note: normal least squares uses same data for y^{val} → biased. the lower variance

Recipe: leave-one-out cross validation to est. ②

Given a sample $y_{i=1,\dots,m}$ & a λ-dependent rule for fitting a model to data, $x_\lambda^{fit}(y_{i=1,\dots,m})$, where λ controls the allowed model complexity, eg

$$x_\lambda^{fit} = \arg\min_x \|y^{obs} - Ax\|^2 + \lambda\|Lx\|^2,$$

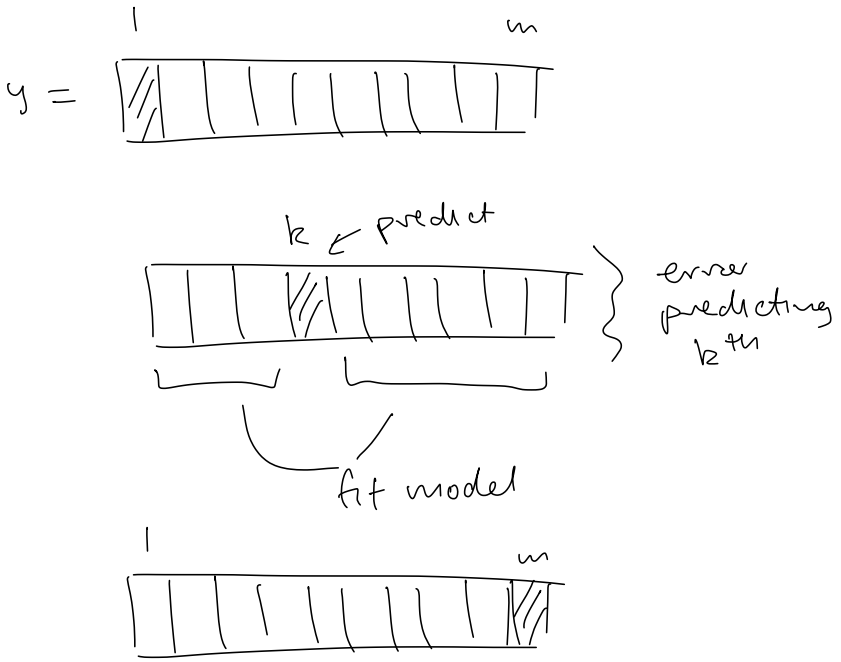
we define $\boxed{x_\lambda^{fit,\, i\neq k}}$ as the model fit on all data except the kth obs y_k & $a_k^T = A[k,:]$ as the observed forward map for the kth observation (k^{th} row)

& estimate the prediction error of the fitting rule by

$$\boxed{Err^{LOOCV}(x_\lambda^{fit}) = \frac{1}{m}\sum_{k=1}^{m}\left(y_k^{obs} - a_k^T x_\lambda^{fit,\, i\neq k}\right)^2}$$

ie the average error from fitting the model on the data with the kth data point removed & predicting the kth data point, for $k = 1,\dots,m$ ie each data point in turn.

Picture:



average error over $k = 1, \ldots m$

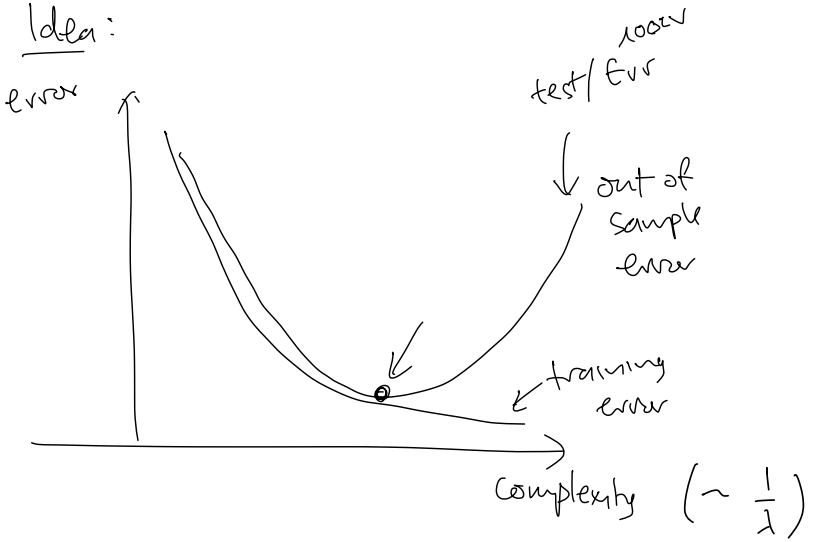
Estimating λ

Given our estimates for each λ, we choose the λ that minimizes

$$\text{Err}^{\text{loocv}}\left(\hat{x}^{\text{fit}}_{\lambda}\right) = \text{Err}^{\text{loocv}}(\lambda)$$

only depends on λ

Idea:



→ typically refit on whole dataset with chosen λ to get final x estimate (this can cause issues!)

Notes

- Simple & general, use for any fitting rule
- unbiased estimate of average predictive error for $m-1$ data points, $\approx$ average predictive error for m data points
- high variance estimate of prediction error !
  - ↳ can improve with eg k-fold cross validation or bootstrap versions
  - K-fold: leave out $\approx m/K$ observations at a time instead of one
    → lower variance, higher bias (typically...)
- less comp. expensive — K fits per λ

Generalised cross validation: cheaper (for linear)

For linear models with linear regularisation, it can be written

$$y^{pred} = A A^{\#}_{\lambda}\, y^{obs}, \quad A^{\#}_{\lambda}: \text{'regularised inverse'}$$

$$= R^{\lambda}_{d}\, y^{obs} \quad \left.\begin{array}{l}\text{data space}\\ \text{pseudo projector}\end{array}\right\}$$

can show a reasonable approximation to $err^{loocv}(\lambda)$ is

$$\frac{1}{m}\frac{\| y^{obs} - A x^{fit}_{\lambda} \|^2}{\left(1 - \text{Trace}(A A^{\#}_{\lambda})/m\right)^2} = \frac{1}{m}\frac{\| y^{obs} - A x^{fit}_{\lambda} \|^2}{\left(1 - \text{Trace}(R_d)/m\right)^2}$$

correction factor

like eg $\sigma^2 = \frac{1}{n-1}\Sigma(y_i - m)^2$ for unbiased variance!

[Taylor series approx. leads to $\approx$ AIC]

See Aster et al Inverse problems
Hastie et al. The elements of statistical learning.

Example: CV attempt

```
lambdas = np.logspace(-3, 1, 260)
L = np.eye(A_poly_obs.shape[1])
cv_error = np.zeros(len(lambdas))
indices = np.arange(len(yp_obs))

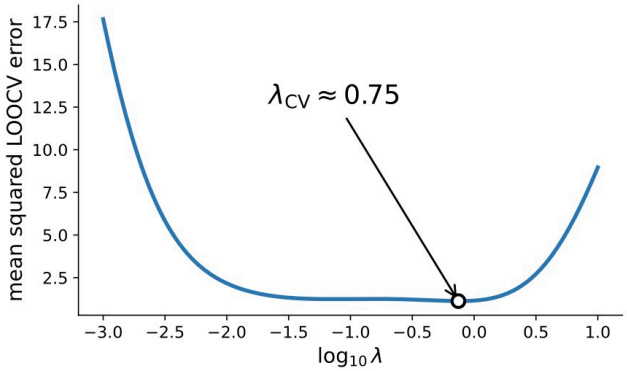
for i, lam in enumerate(lambdas):
    fold_errors = np.zeros(len(yp_obs))
    for k in indices:
        keep = indices != k

        # Fit without observation k
        A_aug = np.vstack([
            A_poly_obs[keep, :], np.sqrt(lam) * L
        ])
        y_aug = np.r_[yp_obs[keep], np.zeros(L.shape[0])]
        theta_k = np.linalg.lstsq(A_aug, y_aug, rcond=None)[0]

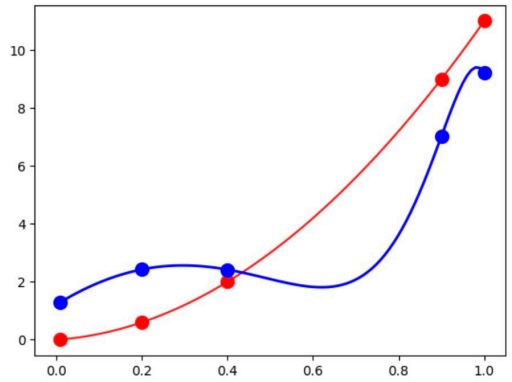
        # Predict the observation that was left out
        error = yp_obs[k] - A_poly_obs[k, :] @ theta_k
        fold_errors[k] = error**2

    cv_error[i] = np.mean(fold_errors)

lambda_cv = lambdas[np.argmin(cv_error)]
```



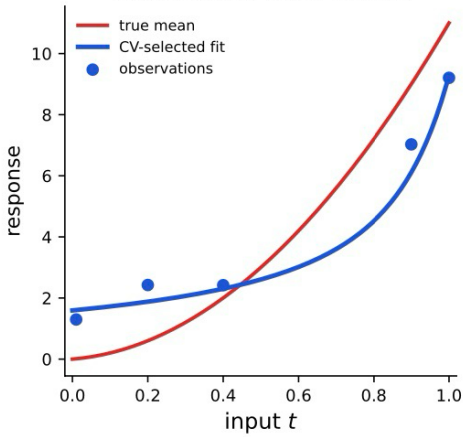
Example cont'd



no regularization

(min norm A^+)

(λ selected then refit to all obs)



Tikhonov + CV

Further reading

- Aster et al Inverse Problems
- Hastie et al The elements of Statistical learning
- Efron (1983) 'Estimating the error rate of a prediction rule'
- Bates et al (2024) 'Cross-validation: what does it estimate & how well does it do it?'

Appendix : Double descent??

Reconciling modern machine-learning practice and the classical bias–variance trade-off

Mikhail Belkin[a,b,1], Daniel Hsu[c], Siyuan Ma[a], and Soumik Mandal[a]

[a]Department of Computer Science and Engineering, The Ohio State University, Columbus, OH 43210; [b]Department of Statistics, The Ohio State University, Columbus, OH 43210; and [c]Computer Science Department and Data Science Institute, Columbia University, New York, NY 10027

Edited by Peter J. Bickel, University of California, Berkeley, CA, and approved July 2, 2019 (received for review February 21, 2019)

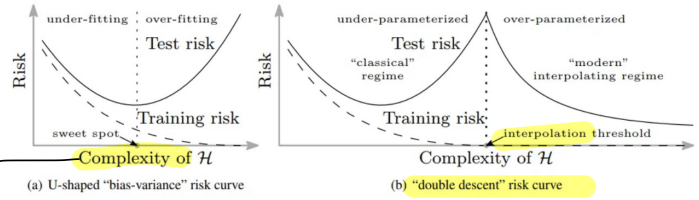


Figure 1: Curves for training risk (dashed line) and test risk (solid line). (a) The classical *U-shaped risk curve* arising from the bias-variance trade-off. (b) The *double descent risk curve*, which incorporates the U-shaped risk curve (i.e., the "classical" regime) together with the observed behavior from using high complexity function classes (i.e., the "modern" interpolating regime), separated by the interpolation threshold. The predictors to the right of the interpolation threshold have zero training risk.

- Some ML researchers have claimed the 'bias-variance' U curves of prediction risk are outdated
- The key issue in my opinion is distinguishing between:

1. dimension of model/data space } varying dim gives double descent
& 2. 'complexity'/'norm' etc of models in a space of given dimension } varying norm for fixed dim → no double descent

ie: 'what is the x axis?'

Appendix: splines fit with pseudo inverse & dd.

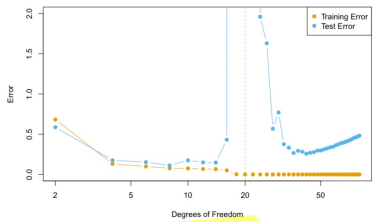


FIGURE 10.20. *Double descent phenomenon, illustrated using error plots for a one-dimensional natural spline example. The horizontal axis refers to the number of spline basis functions on the log scale. The training error hits zero when the degrees of freedom coincides with the sample size $n = 20$, the "interpolation threshold", and remains zero thereafter. The test error increases dramatically at this threshold, but then descends again to a reasonable value before finally increasing again.*

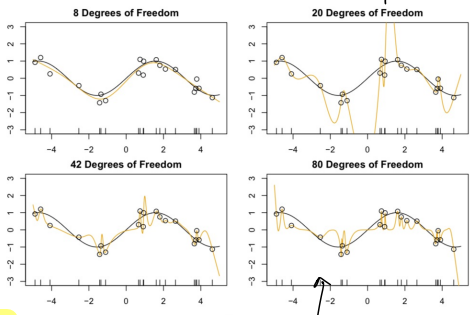


FIGURE 10.21. *Fitted functions $\hat{f}_d(X)$ (orange), true function $f(X)$ (black) and the observed 20 training data points. A different value of d (degrees of freedom) is used in each panel. For $d \geq 20$ the orange curves all interpolate the training points, and hence the training error is zero.*

dimension!

dim

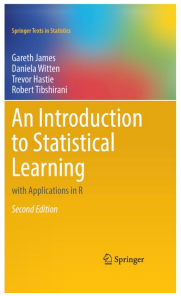
higher dimension but less complex!

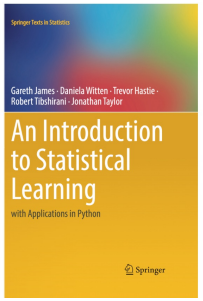
- Some points are worth emphasizing:
  - *The double-descent phenomenon does not contradict the bias-variance trade-off, as presented in Section 2.2.2.* Rather, the double-descent curve seen in the right-hand side of Figure 10.20 is a consequence of the fact that the x-axis displays the number of spline basis functions used, which does not properly capture the true "flexibility" of models that interpolate the training data. Stated another way, in this example, the minimum-norm natural spline with $d = 42$ has lower variance than the natural spline with $d = 20$.

dim

norm

From:





Think: $\min_{x \in \mathbb{R}^d} \|x\|^2$ s.t. $Ax = b$

} $d \uparrow$ means bigger space to find smaller soln!